Student Grade Management System

PCCCS495 – Term II Project Report

Made with GAMMA

Project Overview

A console-based Java application designed to manage student academic records efficiently. The system allows users to add undergraduate and postgraduate students, automatically calculate grades based on predefined grading scales, display all student records, and search for specific students by roll number.

Built using core Object-Oriented Programming (OOP) principles including Abstraction, Encapsulation, Inheritance, Polymorphism, Exception Handling, and Java Collections.

Made with GAMMA

Problem Statement

Secure Storage

Stores student information securely with proper validation

Data Validation

Validates input data to prevent errors and maintain integrity

Automatic Calculation

Calculates grades automatically based on academic level

Quick Retrieval

Allows quick searching and retrieval of student records

Manual grade calculation and student record management is error-prone and inefficient. This project addresses these needs through a well-structured Java OOP application.

Made with GAMMA

Project Objectives

01

Abstract Class Hierarchy

Design an abstract class hierarchy to represent different types of students

02

Exception Handling

Implement custom exception handling for input validation

03

Java Collections

Use Java Collections to manage dynamic lists of student records

04

Polymorphic Behavior

Demonstrate polymorphic behavior through different grading scales

05

Console Interface

Build an interactive console-based user interface

Made with GAMMA

System Architecture

Class Hierarchy

Main

Entry point with interactive menu

GradeController

Manages ArrayList of students

Student (Abstract)

Base class with common fields

Undergraduate

Standard grading rules

Postgraduate

Stricter grading rules

File Structure

File	Purpose	OOP Concept
InvalidGradeException.java	Custom checked exception for invalid marks (0-100)	Exception Handling
Student.java	Abstract base class with private fields, getters/setters, and abstract calculateGrade()	Abstraction, Encapsulation
Undergraduate.java	Concrete subclass with standard grading rules	Inheritance, Polymorphism
Postgraduate.java	Concrete subclass with stricter grading rules and specialization field	Inheritance, Polymorphism
GradeController.java	Controller class with ArrayList for managing student records	Collections
Main.java	Entry point with interactive console menu and input validation	Exception Handling

Made with GAMMA

Detailed Class Descriptions

1

InvalidGradeException

Extends Exception (checked exception). Thrown when student marks are outside valid range (0–100). Contains constructors for custom error messages.

2

Student (Abstract)

Abstract base class with private fields (name, rollNumber, marks). Constructor validates marks using InvalidGradeException. Abstract method calculateGrade() must be implemented by subclasses.

3

Undergraduate

Extends Student with standard grading scale: 75-100 (A), 60-74 (B), 50-59 (C), 40-49 (D), below 40 (F).

4

Postgraduate

Extends Student with stricter grading scale: 80-100 (A+), 70-79 (A), 60-69 (B), 50-59 (C), below 50 (F). Includes specialization field.

5

GradeController

Manages ArrayList of students with methods to add, display all, and search by roll number.

6

Main

Entry point with menu-driven interface and comprehensive input validation using try-catch blocks.

Made with GAMMA

OOP Concepts Demonstrated

Abstraction

Student declared as abstract class defining structure. Abstract method calculateGrade() forces subclasses to provide implementation.

Encapsulation

Private fields with public getters/setters. Validation logic prevents invalid values from external code.

Inheritance

Undergraduate and Postgraduate extend Student, inheriting fields and methods. Constructors use super() to call parent.

Polymorphism

calculateGrade() has different implementations. Java automatically picks correct version based on object type at runtime.

Exception Handling

InvalidGradeException extends Exception. Caught in Main.java with try-catch blocks for graceful error handling.

Collections

ArrayList in GradeController stores student records with dynamic sizing. Holds both Undergraduate and Postgraduate objects.

Made with GAMMA

Sample Output

Adding Students

```
==========================================
STUDENT GRADE MANAGEMENT SYSTEM
==========================================
1. Add Undergraduate Student
2. Add Postgraduate Student
3. Display All Students
4. Search by Roll Number
5. Exit
----------------------------------------
Enter your choice: 1
Enter name: Pranjal Gupta
Enter roll number: UG001
Enter marks (0-100): 85
Student added: Pranjal Gupta (UG001)
```

Display All Students

```
Enter your choice: 3
----------------------------------------
[Undergraduate] Name: Pranjal Gupta
| Roll Number: UG001
| Marks: 85.0
Grade: A
[Postgraduate] Name: Neha Verma
| Roll Number: PG001
| Marks: 92.0
| Specialization: AI
Grade: A+
----------------------------------------
```

Search by Roll Number

```
Enter your choice: 4
Enter roll number to search: pg001
[Postgraduate] Name: Neha Verma
| Roll Number: PG001
| Marks: 92.0
| Specialization: AI
Grade: A+
```

Error Handling

```
Enter your choice: 1
Enter name: Test
Enter roll number: T001
Enter marks (0-100): 105
Error: Invalid marks: 105.0
Marks must be between 0-100.

Enter marks: hello
Error: Marks must be a number.
```

Made with GAMMA

Conclusion

The Student Grade Management System successfully demonstrates all core OOP concepts required for the Term II project.

Robust Implementation

Handles all edge cases gracefully with comprehensive error handling

Clean Interface

Provides interactive user experience through console menu

Well-Structured

Uses OOP principles ensuring maintainable and extensible codebase

Made with GAMMA

Thank You

Student Grade Management System

PCCCS495 – Term II Project

Made with GAMMA